



AQI Forecast MLOps Documentation & Implementation

Submitted By

Name: Naveed Jokhio

Live Dashboard: aqi-predictor-dashboard-444682761540.asia-south1.run.app

Submission Date: 02/09/2026

Academic Session: 6th

Table of Contents

0.0	Executive Summary	03
1.0	Problem & Solution Engineering	03
2.0	Data Layer Architecture.....	04
3.0	Model & Experimentation.....	04
4.0	Production Operations	05
5.0	Monitoring, Explainability & Governance	05
6.0	Security & Compliance	06
7.0	Live Deployment & Results.....	06
8.0	Engineering Decisions - Lessons Learned	06

Artifact & Deployment Catalog

Artifact / Service	Platform	Location / URI
Feature Store	BigQuery	Feature tables per city, refreshed hourly
Model Registry	Vertex AI	One registry entry per city, versioned
Dashboard Service	Cloud Run	aqi-predictor-dashboard-444682761540.asia-south1.run.app
REST API Service	Cloud Run	aqi-predictor-api-444682761540.asia-south1.run.app
CI/CD Pipelines	GitHub Actions	.github/workflows/ (hourly features, daily retrain)
Source Code	GitHub	github.com/Naveedjokhio/10Pearls_aqi-predictor

0.0 Executive Summary

The 10Pearls AQI Predictor is a 100% serverless, end-to-end MLOps pipeline built during a Data Sciences Internship at 10Pearls Pakistan. The brief called for a 3-day Air Quality Index forecast for Karachi; the implementation was extended to cover six major Pakistani cities, turning a single-city notebook exercise into a scalable, production-grade forecasting service.

The system automates the full lifecycle - data collection, feature engineering, model training, evaluation, deployment and retraining - with no manual intervention required once deployed.

Business Alignment

- **Goal:** Deliver actionable AQI forecasts up to 3 days ahead for six Pakistani cities.
- **Model quality bar:** Each city/horizon model is evaluated on RMSE, MAE and R^2 ; the final forecast is the median of three independently trained model families, which limits the impact of any single model's outliers.
- **Operational bar:** Fully automated hourly feature refresh and daily retraining via GitHub Actions, with both the dashboard and API deployed as independent, always-available Cloud Run services.
- **Cost bar:** Every layer of the stack - feature store, model registry, compute - is serverless and scales to zero when idle.

1.0 Problem & Solution Engineering

Three roadblocks surfaced while moving from a research notebook to a production pipeline. Each was investigated, evidenced, and resolved using an approach already permitted by the project brief - rather than a workaround.

#	Problem	Solution
1	No live ground-station data for Karachi/Sindh - AQICN's own station-search and map APIs confirmed no active station covering the region.	Switched to OpenWeather's satellite-based Air Pollution API, which gave reliable, current coverage for all six target cities plus real hourly historical data for training.
2	The originally planned feature store, Hopsworks, was reporting an active platform outage affecting data ingestion at the time of setup.	Pivoted to BigQuery as the feature store and Vertex AI Model Registry for trained models - the alternative explicitly permitted by the project brief ("Hopsworks or Vertex AI").
3	Manually re-running scripts meant no schedule, no failure handling, and forecasts going stale within hours - no automation, no trust.	Automated the whole pipeline with GitHub Actions: hourly feature collection and daily model retraining, authenticated keylessly via Workload Identity Federation instead of static service-account keys.

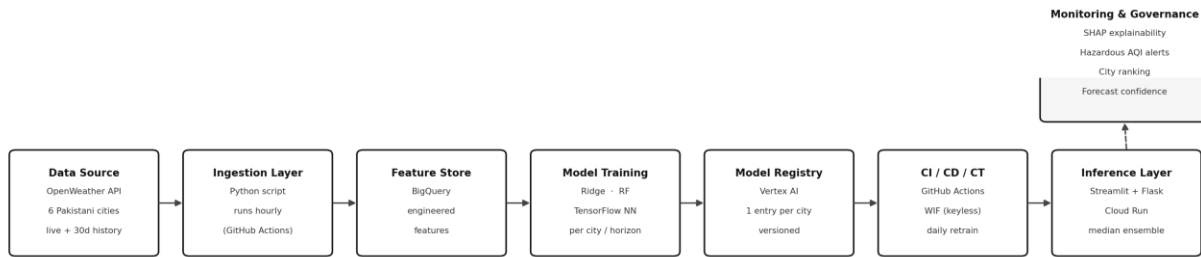
This pattern - investigate, gather evidence, apply an already-approved alternative - is the same engineering discipline documented in Section 8, and is treated as a design decision rather than a deviation from the brief.

2.0 Data Layer Architecture

Sources & Ingestion

- **Source:** OpenWeather API - live weather and pollutant concentrations for six cities, plus roughly 30 days of real hourly historical data used for model training.
- **Ingestion:** A Python feature-generation script, triggered hourly by GitHub Actions, computes time-based features (hour, day, month) and derived features such as AQI change rate.
- **Feature Store:** BigQuery holds the engineered features, keyed by city and timestamp, and serves both training and inference.
- **Model Registry:** Vertex AI Model Registry holds one versioned entry per city (and forecast horizon), so every deployed model can be traced back to its training run.

End-to-End Architecture



Six cities · 100% serverless — every component scales to zero when idle

Six cities run through the identical pipeline in parallel; every component - ingestion, feature store, training, registry, CI/CD and inference - is serverless and scales to zero between scheduled runs.

3.0 Model & Experimentation

Three independent model families are trained for every city and every forecast horizon (Day 1, Day 2, Day 3):

- **Ridge Regression** (scikit-learn) - a fast, regularised linear baseline.
- **Random Forest** (scikit-learn) - captures non-linear relationships between weather and pollution features.
- **TensorFlow Neural Network** - a custom deep model competing directly against the statistical models.

Each model is scored on RMSE, MAE and R^2 . Rather than picking one “winning” architecture, the final forecast is the median of all three predictions per city and horizon - a simple ensembling approach that guards against any single model's outliers.

Sample Results - Karachi

Karachi is shown below as a representative sample; the same training and evaluation pipeline runs independently for all six cities.

Horizon	Best Model	RMSE	R^2
Day 1	Neural Network	6.9	0.75
Day 2	Random Forest	7.8	0.66

Horizon	Best Model	RMSE	R ²
Day 3	Neural Network	7.0	0.72

Accuracy is naturally highest for the Day 1 horizon and gradually loosens further out - the expected pattern for a weather-driven forecasting problem - which is why each horizon is trained and evaluated as its own model rather than a single multi-step forecaster.

4.0 Production Operations

CI / CD / CT Pipeline Design

- **Continuous Integration:** Feature-generation and training scripts run on every push as part of the GitHub Actions workflow.
- **Continuous Deployment:** Validated models and application code are deployed to Cloud Run as independent services.
- **Continuous Training:** Feature collection runs hourly; full model retraining runs daily, so every model is refreshed against the latest available data without manual intervention.

Authentication & Deployment

- **Workload Identity Federation:** GitHub Actions authenticates to Google Cloud keylessly - no long-lived service-account keys are stored in the repository or the runner.
- **Two independent Cloud Run services:** a Streamlit dashboard (interactive UI, city selection, SHAP explainability, hazardous-AQI alerts) and a Flask REST API exposing the same forecasts as JSON - deployed and scaled separately.
- **100% serverless:** every component - ingestion, feature store, registry, dashboard and API - scales to zero and incurs no cost while idle.

5.0 Monitoring, Explainability & Governance

Explainability

The dashboard integrates SHAP feature-importance so any single forecast can be traced back to the features that drove it, rather than being produced by a black box.

Operational Monitoring

- **Hazardous AQI alerts:** threshold-based warnings surface directly on the dashboard when a forecast crosses into an unhealthy range.
- **City ranking:** the six cities are compared side-by-side, making relative air-quality risk visible at a glance.
- **Forecast confidence indicators:** surfaced alongside each prediction so users can gauge how much to trust a given number.
- **Drift mitigation:** rather than a separate drift-detection service, daily retraining keeps every model current against the latest 30-day window of real data - the primary defence against concept drift for this deployment.

- **Raw-data export:** underlying features and predictions can be exported directly from the dashboard for independent auditing.

6.0 Security & Compliance

- **No hardcoded credentials:** API keys and cloud credentials are never committed to source; secrets are injected at run time.
- **Keyless CI/CD:** Workload Identity Federation removes the need for static GCP service-account keys inside GitHub Actions, reducing the blast radius of a leaked secret to effectively zero.
- **Centralised lineage:** BigQuery as the single feature store means every prediction can be traced back through its feature row to the raw OpenWeather reading that produced it.
- **Versioned models:** Vertex AI Model Registry keeps a full, timestamped history of every model trained per city, so a deployed model can always be matched back to its training run.

7.0 Live Deployment & Results

Live Links

- **Dashboard:** aqi-predictor-dashboard-444682761540.asia-south1.run.app
- **API:** aqi-predictor-api-444682761540.asia-south1.run.app
- **Source code:** github.com/Naveedjokhio/10Pearls_aqi-predictor

Both services are public, run 24/7 independent of any local machine, and are backed entirely by managed, serverless infrastructure.

Scope Delivered

- Multi-city coverage: six major Pakistani cities, versus the single-city (Karachi) brief.
- Three model families per city per forecast horizon, combined via median ensembling.
- Fully automated feature refresh (hourly) and retraining (daily).
- Two independently deployed, serverless production services: dashboard and API.
- SHAP explainability, hazardous-AQI alerting, city ranking, confidence indicators and raw-data export.

8.0 Engineering Decisions - Lessons Learned

The most valuable part of this project was working within real-world constraints rather than around them. Two decisions stand out:

Decision 1: Data Source Substitution

AQICN was the natural first choice for ground-truth air-quality readings, but its own station-search and map APIs confirmed there was no active monitoring station for Karachi or Sindh. Rather than blocking on unavailable data, the project moved to OpenWeather's satellite-based Air Pollution API, which offered verified, consistent coverage across all six target cities.

Decision 2: Feature Store Substitution

Hopsworks was the platform named first in the project brief, but it was independently confirmed to be experiencing a data-ingestion outage at the time of setup. Since the brief explicitly permitted Vertex AI as an alternative, the project adopted BigQuery as the feature store and Vertex AI Model Registry for trained models - a documented, permitted substitution rather than a shortcut.

Takeaway

Both decisions followed the same discipline: confirm the problem with evidence before working around it, and prefer an already-approved alternative over an improvised one. That discipline - more than any single model's accuracy - is what let the system reach a stable, 24/7 production deployment.